Sistema distribuito

Consideriamo ora il seguente sistema distribuito:

- Fissato un insieme di n nodi $[n] = \{1, 2, \dots, n\}$
- Ogni nodo può eseguire sia computazioni arbitrarie che inviare messaggi di lunghezza arbitraria ad ogni altro nodo

All'interno di questo sistema consideriamo ora il problema del Byzantine Broadcast dove gli n nodi si vogliono mettere d'accordo su quale bit dare in output

Byzantine broadcast

Def

Sia $V = \{1, \dots, n\}$ un insieme di nodi t.c. $|V| = n$ e V costituisce un sistema distribuito

Sia $F \subseteq V$ un sottoinsieme di V composto da $|F| = f$ nodi "corrotti", con $0 \leq f < n$

Sia $1 = s \in V$ nodo sorgente al quale viene affidato un messaggio b

Si vuole progettare un protocollo al termine del quale $\forall i \notin F$ nodo onesto, questo dia in output un valore y_i t.c. valgano:

A) Consistency:

$\forall i, j \notin F \quad y_i = y_j$

B) Validity: Se la sorgente è onesta allora tutti i nodi onesti rispondono con il messaggio inviato dalla sorgente

Se $s \notin F$ allora $y_i = b \quad \forall i \notin F$

$\forall i \notin F$, i applicherà il nostro protocollo

$\forall j \in F$, j non seguirà il nostro protocollo, assumiamo quindi che questi si coalizzeranno per far fallire il protocollo

Non conosciamo chi sono i nodi corrotti

Assunzioni per il modello

Prima di progettare un protocollo per il BB bisognerà fare prima delle assunzioni sul sistema

a) Sistema sincrono/asincrono:

Assumendo sia garantito che i messaggi arrivino sempre dobbiamo fare ulteriori assunzioni sull'upperbound sul tempo di arrivo dei messaggi

- Un sistema sincrono ci pone in un sistema regolato da un global clock che scandisce il tempo in intervalli discreti t.c. ogni messaggio inviato al tempo t arriva sicuramente al tempo $t+1$
- Un sistema asincrono non ci dà alcun vincolo sull'upperbound sul tempo di arrivo dei messaggi

b) Protocolli autenticati/non autenticati

Dato un mittente i ed un destinatario j dobbiamo fare delle assunzioni sulla certezza di j che il mittente sia realmente i

- Un Protocollo autenticato è ad esempio è il public key infrastructure (PKI) setup:
  - $\forall i \in V$ i ha una coppia di chiavi (SK_i, PK_i) ⇝ si sa sempre chi ha inviato il messaggio
  - L'insieme delle chiavi pubbliche $\{PK_i : i \in V\}$ è noto a tutti i nodi a priori
- Un protocollo non autenticato non dà alcuna garanzia di questo tipo

c) Sistemi permissioned/permissionless

- Un sistema permissioned non permette l'aggiunta e la rimozione di nodi dal sistema
- Un sistema permissionless permette a qualsiasi nodo di entrare e/o uscire dal sistema

↳ Il modello su cui è costruito bitcoin è permissionless

Esercizi

Assumendo che il modello sia

- Sincrono
- Autenticato con pub key infrastructure
- Permissioned

1.a) Mostrare un protocollo banale che soddisfi consistency ma non validity

Protocollo 1.a

- Passo 0

  La sorgente (1) riceve in input $b \in \{0,1\}$

- Passo 1

  La sorgente invia il messaggio $\langle b \rangle_1$ a tutti i nodi

- Passo 2

  $\forall i \in V$ i invia il messaggio $\langle 0 \rangle_i$

Questo protocollo soddisfa consistency in quanto ogni nodo $i \in V \setminus F$ risponderà sicuramente 0

Invece non vale validity visto che se $1 \in V \setminus F$ non vale che $\langle y_i \rangle_i = \langle b \rangle_1$

1.b) Mostrare un protocollo banale che soddisfi validity ma non consistency

Protocollo 1.b

- Passo 0

  La sorgente (1) riceve in input $b \in \{0,1\}$

- Passo 1

  La sorgente invia il messaggio $\langle b \rangle_1$ a tutti i nodi

- Passo 2

  $\forall i \in V$ i invia il messaggio $\langle b \rangle_{1,i}$

Questo protocollo soddisfa validity in quanto se $1 \in V \setminus F$ tutti i nodi invieranno $\langle y_i \rangle_i = \langle b \rangle_1$

Invece non vale consistency visto che se $1 \in F$ può inviare messaggi diversi ai nodi e quindi i nodi onesti risponderanno con dei messaggi differenti

2) Mostrare un protocollo per cui valgano validity e consistency nel caso in cui l'insieme dei nodi corrotti sia noto

Protocollo 1.b

- Passo 0

  La sorgente (1) riceve in input $b \in \{0,1\}$

- Passo 1

  La sorgente invia il messaggio $\langle b \rangle_1$ a tutti i nodi

- Passo 2

  $\forall i \in V$ i invia il messaggio $\langle b \rangle_{1,i}$ a tutti gli altri nodi $j \neq F$ e ricevono un messaggio $\langle b \rangle_{1,j}$

- Passo 3

  $\forall i \in V$ i legge tutti i messaggi $\langle b \rangle_{1,j}$ ricevuti

  if $\exists \langle b \rangle_{1,j} \neq \langle b \rangle_1$

  return $\langle 0 \rangle_i$

  else

  return $\langle b \rangle_{1,i}$

In questo caso valgono:

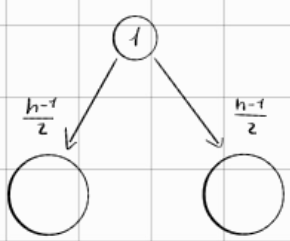
Validity: se $1 \notin F$ tutti i nodi riceveranno il messaggio $\langle b \rangle_1$ e al passo 3 $\forall i$ i riceverà lo stesso messaggio $\langle b \rangle_{1,j}$

Consistency: Con il secondo controllo risolvo il problema del caso $1 \in F$ in quanto $\forall i \notin F$ questo risponde $\langle 0 \rangle_i$ se 1 ha inviato messaggi differenti.

3) Mostrare perché questi due protocolli non vanno bene

Algorithm 1 Tentativo 1

ROUND 0. La sorgente (il nodo 1) riceve in input b.
ROUND 1. La sorgente invia $\langle b \rangle_1$ a tutti i nodi.
ROUND 2. Ogni nodo i:
  se nel ROUND 1 ha ricevuto $\langle \hat{b} \rangle_1$, invia $\langle \hat{b} \rangle_i$ a tutti i nodi;
  altrimenti invia $\langle 0 \rangle_i$ a tutti i nodi.
ROUND 3. Ogni nodo i:
  se c'è un unico valore $\hat{b}$ per cui nel ROUND 2 ha ricevuto più di $n/2$ messaggi $\langle \hat{b} \rangle_j$, restituisce in output $\hat{b}$;
  altrimenti restituisce in output 0.



Questo non funziona perché $\forall i \in F$ è possibile che questi mandino messaggi diversi a tutti gli altri nodi

Algorithm 2 Tentativo 2

ROUND 0. La sorgente (il nodo 1) riceve in input b;
ROUND 1. La sorgente invia $\langle b \rangle_1$ a tutti i nodi;
ROUND 2. Ogni nodo i
  se nel ROUND 1 ha ricevuto $\langle \hat{b} \rangle_1$, invia $\langle \hat{b} \rangle_i$ a tutti i nodi;
  altrimenti invia $\langle 0 \rangle_i$ a tutti i nodi.
ROUND 3. Ogni nodo i
  se nel ROUND 2 ha ricevuto $\langle \hat{b} \rangle_j$ da ogni altro nodo j, restituisce in output $\hat{b}$;
  altrimenti restituisce in output 0.

DRAFT